

BOOTCAMP FRONTEND & REACT · BLOQUE REACT · CLASE 2

Las piezas se comunican

Props opcionales, composición y la regla de oro — y de largo: el catálogo que se pinta solo y la página que decide.

4 h 15 · L·M·V

Retomamos donde cortamos

STEER
ROCK
LA

SIN REPASOS LARGOS: RETOMAMOS Y SEGUIMOS

El plan de hoy

0. Reapertura

retomar el flujo donde se cortó

10'+

1. Props opcionales

? y = · el Header de la tienda

20'

2. Composición

Footer · el árbol · DevTools · la regla de oro

25'

3. Leer App + build

la página como índice

10'

4+ FLUJO: listas y condicionales

map · key · ternario · && · estados

→ 3:50

C. Cierre + 5 tareas

push demostrado, dudas y anticipo

25'

**Receso de 15 min** alrededor de las 2:00 · verde = **flujo: se corta donde toque y la clase 3 retoma exacto**

El problema, en sus palabras

"Si cambio el diseño de la tarjeta, tengo que corregirlo tres veces."

— comentario de la tarea

"Es fácil equivocarse en una de las copias y que quede distinta."

— comentario de la tarea

"Con cuarenta productos esto sería imposible de mantener."

— comentario de la tarea

Diagnosticaron el problema solos. La solución ya la tienen: **la tarjeta se escribe una vez** y cada uso recibe su producto por props.

PROPS QUE PUEDEN FALTAR

? opcional • = por defecto

```
interface Props {  
  nombre: string;  
  eslogan?: string; // puede faltar - el ? de TypeScript  
}  
  
function Header({ nombre, eslogan = 'Tecnología para todos' }: Props)
```

```
<Header nombre="Tienda Tec" />
```

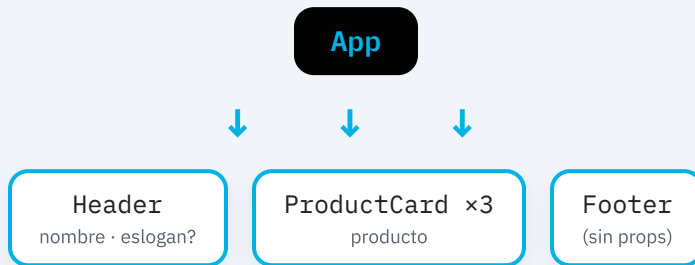
sin eslogan → sale el valor por defecto

```
<Header />
```

Property 'nombre' is missing — lo obligatorio se exige, en el editor

COMPOSICIÓN · UNA PÁGINA ES UN ÁRBOL DE PIEZAS

Los datos **bajan**



De padre a hijo, en **un solo sentido**. App conoce el catálogo; la tarjeta solo conoce SU producto.

Si algo se ve mal, el dato malo vino de arriba: **un solo camino que revisar**.

LA REGLA DE ORO DE REACT

Las props **no se tocan**

```
// dentro de ProductCard:  
producto.precio = 0; // compila... y rompe la regla
```

Un componente **lee** sus props, las **muestra**, y nada más — la disciplina de las **funciones puras** de fundamentos. Si la tarjeta modifica el producto, App ya no puede confiar en su propio dato.

¿Y cómo se cambian los datos **legalmente**? Esa pregunta ES la clase 4 (useState). Hoy: se mira, no se toca.

El catálogo **se pinta solo**

```
<ProductCard producto={productos[0]} />  
<ProductCard producto={productos[1]} />  
<ProductCard producto={productos[2]} />  
  
{productos.map((p) => (  
  <ProductCard producto={p} />  
))}
```

El map de fundamentos (el de los Streams de Java) + las llaves de JSX: si la expresión devuelve un **array de JSX**, React pinta cada elemento.

Agregar un producto a datos.ts = agregar una tarjeta. Nadie vuelve a tocar este JSX.

EL PEAJE DEL MAP · EACH CHILD SHOULD HAVE A UNIQUE "KEY" PROP

key = identidad

`key={1}`
Audífonos

`key={2}`
Laptop 14"

`key={3}`
Reloj

`key={p.id}`
siempre el id del dato

```
{productos.map((p) => <ProductCard key={p.id} producto={p} />)}
```

`key={p.id}`

la identidad viaja con el dato: filtrar y reordenar no la rompen

`key={indice}`

la posición cambia cuando la lista cambia — **la identidad miente**

CONDICIONALES: TERNARIO · && · RETURN NULL

Qué pinta JSX — y la trampa del 0

false · null · undefined

JSX los traga en silencio — por eso el && funciona

0 · ' ' · textos · números

Sí se pintan — y ahí está la trampa

```
{producto.stock && <p>...</p>} // stock = 0 → pinta un 0 suelto
{producto.stock > 0 && producto.stock <= 5 && <p>¡Últimas unidades!</p>}
```

Regla del proyecto: con números, la condición del && es una **comparación explícita**. Nunca el número pelado.

EstadoCarga: la página sabe qué decir

```
export type EstadoCarga = 'cargando' | 'listo' | 'error';
```

'cargando'

Cargando catálogo...

'listo'

las tarjetas, con el map

'error'

No pudimos cargar los productos

Hoy la constante se cambia **a mano** y la pantalla obedece. Que cambie sola necesita useState (clase 4) y datos reales (clase 7). **La pantalla ya sabe qué hacer cuando eso llegue.**

HASTA DONDE LLEGAMOS HOY

Lo de hoy

? y =

props opcionales con valor por defecto — lo obligatorio se exige

composición

componentes dentro de componentes; cada pieza en su archivo

los datos bajan

de padre a hijo, un solo sentido — y las **props no se tocan**

DevTools

el árbol, las props, y UI = f(datos) visto en vivo

map + key

el catálogo se pinta solo; la key es el id, nunca el índice

condicionales

ternario · && con comparación explícita · return null

CRITERIO DE ENTREGA: NPM RUN BUILD SIN ERRORES + PUSH

5 tareas

FÁCIL

Replica la clase, hasta donde llegamos: tu tienda al día — Header con nombre y eslogan, Footer, y lo que el flujo haya alcanzado. Con la grabación al lado. Es LA tarea.

INTERMEDIO

Los beneficios de tu tienda: componente `Beneficio` con `titulo` (obligatorio) y `detalle?` (con valor por defecto), usado tres veces: envío, cuotas, garantía.

INTERMEDIO

El precio como pieza: componente `Precio` (recibe `precio: number`, muestra base y con IGV usando tu `formatearPrecio`) dentro de `ProductCard`. Composición.

DIFÍCIL

El catálogo con categorías: amplíalo a 12 productos y suma `categoria: string` al contrato, propagado a todos. TypeScript te guía. Es la semilla de los filtros (clase 5).

DIFÍCIL

Por qué el índice miente: lee "Renderizar listas" en es.react.dev y explica en un comentario largo, con tus palabras, por qué el índice como `key` se rompe al filtrar o reordenar.

Próxima clase — retomamos donde cortamos

Los botones cobran vida

```
// hasta hoy, TODO se cambia editando el archivo...  
<button onClick={agregarAlCarrito}>Agregar al carrito</button>  
  
// ...y la pieza que llevamos dos clases rodeando:  
const [carrito, setCarrito] = useState<ItemCarrito[]>([])
```

Eventos — el primo del **addEventListener** de fundamentos — y **useState**: la memoria del componente. El contador que leíste en la tarea de la clase 1 por fin queda explicado.

El 🛒 0 del encabezado va a dejar de mentir. Trae las tareas hechas y el build en cero.